

Can an AI Reader Beat Geometry?

Benchmarking Freedi's claim registry against text embeddings on 875 adversarial statement triplets

Freedi Research · 16 July 2026 · Branch `feat/claim-registry-improvements` · Evaluation data: Blair, Procaccia & Tambe, *Embeddings for Preferences, Not Semantics* (2026)

Abstract. Deliberation platforms must decide, for every incoming statement, whether it repeats an idea already on the table, contradicts one, or introduces something new. The standard technique — measuring the distance between text *embeddings* — has a documented blind spot: it confuses *similar wording* with *similar opinion*. We tested Freedi's alternative, the **claim registry** (a small language model that reads a list of existing claims and judges each new statement against them), on the hardest published test of this blind spot: 875 triplets in which the correct partner statement uses different words and the wrong one uses nearly identical words with the opposite stance.

The registry's production model (gpt-4o-mini) resolved **95.0%** of triplets correctly. The embedding model used in Freedi's own pipeline resolved **0.5%**, and the reference paper's best system — an embedding model fine-tuned specifically for this task — reached **80.0%**. However, when the registry judged against the short AI-written claim *summaries* that production actually stores, accuracy fell to **70.1%**: the weak link is not the judge but the summarized text it is given. We conclude with the production changes this implies and the run's operational lessons.

1 The problem in one example

Suppose one participant has already written:

EXISTING STATEMENT (ANCHOR) “It's so high that it will definitely cost some needy people their jobs.”

Two new statements arrive:

NEW STATEMENT 1 — SAME OPINION, DIFFERENT WORDS (THE “MATCH”) “The increase is so substantial that it is bound to lead to job losses among those in need.”

NEW STATEMENT 2 — NEARLY IDENTICAL WORDS, OPPOSITE OPINION (THE “DISTRACTOR”) “It's so high that it will definitely **provide** some needy people **with job opportunities**.”

A deliberation platform should file statement 1 *together with* the anchor and keep statement 2 *apart* (ideally noting that it disagrees). This is harder than it looks for machines. The dominant technique converts each sentence into a point in a high-dimensional space — an **embedding** — and treats small distances as “same idea.” But embeddings are dominated by topic and vocabulary. Statement 2 shares

almost every word with the anchor, so it sits *closer* in embedding space than statement 1, which says the same thing in fresh words. The geometry files the disagreement together with the opinion it disagrees with, and exiles the honest paraphrase.

Blair, Procaccia and Tambe (Harvard) formalized this failure and published an evaluation set of 875 such **hard triplets** — (anchor, match, distractor) — drawn from eleven real public deliberations (Polis conversations on the Seattle minimum wage, Brexit, Canadian electoral reform, universal basic income, and a Kentucky town plan; Remesh sessions on campus protests, foreign intervention, and the right to assemble; and three generative-social-choice surveys on abortion policy and chatbot personalization). The anchors are real participant statements; the match and distractor are controlled rewrites. A system scores a triplet correct only if it accepts the match *and* rejects the distractor. Off-the-shelf embedding models score between 6% and 48% on this test — worse than coin-flipping — and the paper's own remedy, an embedding model fine-tuned on synthetic stance data (“DPT”), reaches 80.0%.

Freedi's **claim registry** takes a different route entirely: instead of improving the ruler, it hires a reader.

2 What the claim registry is

In Freedi, every question accumulates a **codebook**: a short list of *claims*, one per distinct idea participants have raised (“Add protected bike lanes on main streets”). When a new statement arrives and the fast geometric checks cannot place it, the registry makes **one call to a small, inexpensive language model** (gpt-4o-mini). The model is shown three things — the question, the numbered list of claims, and the new statement — and must answer in a fixed format: does the statement **express** one of the claims (same proposal in any words), **oppose** one (the contrary proposal on the same subject), or **none**? It also reports a confidence; production only files a statement under a claim when the answer is “expresses” with confidence ≥ 0.6 . An “opposes” verdict is never filed as a match — it is recorded as a *counter-edge*, giving the synthesis layer explicit pro/con structure.

Crucially, the model never sees other participants' raw statements and never compares statements to each other — only the short claim list and one new statement per call. Cost therefore grows with the number of distinct *ideas*, not the number of *participants*: about \$0.11 per thousand statements at typical codebook sizes.

3 What we tested — the six conditions

All conditions answer the same 875 triplets; they differ only in *which mechanism decides*. The benchmark calls Freedi's production code directly — nothing was re-implemented for the test.

Condition A — Embeddings only (today's pipeline without the registry)

Each statement becomes a point using the exact embedding model Freedi's pipeline uses (`text-embedding-3-small`), and cosine distance decides. A triplet passes if the match lands closer to the anchor than the distractor. We also simulated the pipeline's real attachment thresholds (join a cluster at similarity ≥ 0.60 ; ≥ 0.85 for the stricter first pass) to ask: what would production do *without* the registry?

Condition A2 — Embeddings shortlist, then the reader (the standard “RAG” pattern)

The common industry design: use cheap geometry to *shortlist* candidate claims, and let the language model judge only the shortlist. If geometry never shortlists the right claim, the reader never sees it. Freedi's registry deliberately skips the shortlist and always shows the full claim list; this condition measures what that choice is worth here.

Condition B1 — The registry reading the participant's own words

The registry classifier exactly as designed, in the cleanest setting: the codebook contains one claim — the anchor statement verbatim — and the model judges the match and the distractor against it in separate calls, with the production confidence rule applied. This isolates the pure judging ability of the mechanism.

Condition B2 — The registry reading the AI-written summary (production-faithful)

Identical to B1 with one change: the claim text is not the anchor's own words but the 5–15-word neutral summary that production's claim-generation step writes when a claim is born — because that summary is what real codebooks store. Comparing B1 to B2 measures how much meaning the summarization step loses.

Condition C — The registry with a realistic, crowded codebook

Real questions have dozens of claims, not one. Here the codebook contains the generated claims of *all* anchors from the same deliberation (13–100 claims, average 94), ordered exactly as production orders them. The model must pick the right claim out of the crowd. One caveat known in advance: this benchmark codebook never ran production's *consolidation* step, which merges duplicate claims — so many equivalent claims coexist, and matching an equivalent-but-different claim is (unfairly) scored as an error.

Condition D — Same as B1, with the expensive model

gpt-4o ($\approx 20\times$ the price of gpt-4o-mini) replaces the worker model, answering whether production is leaving accuracy on the table by using the cheap model.

Method notes: a stratified 150-triplet pilot (fixed seed, proportional across the 11 source deliberations) served as a go/no-go gate before the full run. All rates carry Wilson 95% confidence intervals; paired comparisons use exact McNemar tests. The triplet files do not include the original question text, so per-deliberation questions were reconstructed from the paper's dataset descriptions.

4 Results

System (n = 875 triplets)	Triplet accuracy	95% CI
A — Embeddings only (Freedl's production embedding model)	0.5%	[0.2, 1.2]
<i>Best off-the-shelf encoder in the reference paper (sentence-T5-XL)</i>	48.3%	—
<i>Reference paper's fine-tuned system (DPT sentence-T5-XL)</i>	80.0%	—
B2 — Registry reading AI-written claim summaries (production-faithful)	70.1%	[66.9, 73.0]
B1 — Registry reading the participant's own words (gpt-4o-mini)	95.0%	[93.3, 96.2]
A2 — Embedding shortlist + reader (gate 0.45 or 0.60)	95.0%	identical to B1
D — Registry, participant's own words, gpt-4o	96.5%	[95.0, 97.5]
C — Registry, 94-claim codebooks, strict own-claim scoring	36.0%	[32.9, 39.2] (§4.4)

Table 1 — Headline triplet accuracy. Statistical strength of the central contrasts (exact McNemar): registry (B1) vs embeddings (A): 827 vs 0 discordant triplets, $p \approx 10^{-249}$; B1 vs B2: 222 vs 4, $p \approx 10^{-60}$; B1 vs D: 12 vs 25, $p = 0.047$.

4.1 Embeddings cannot see disagreement — and production thresholds make it worse

The production embedding model resolved 4 triplets out of 875. This is below every encoder in the reference paper: the stance flip barely moves the vector because the vector is made of topic and vocabulary, which the flip shares by construction. The threshold simulation turns this into an operational statement: **100%** of the stance-flipped distractors score above the pipeline's cluster-joining threshold (0.60) against the anchor, and 99.1% clear even the strict 0.85 threshold. Without the registry pass, essentially every “I oppose X” arriving after “I support X” would be merged into the same cluster. No threshold can fix this — on these pairs the flip is *always* at least as close as the honest paraphrase.

4.2 The reader beats the ruler — including the purpose-built ruler

Reading the claim and judging meaning (B1) resolved 831 of 875 triplets: 96.8% of matches accepted, only 2.1% of distractors wrongly accepted — and 95.9% of distractors explicitly recognized as *opposing* the claim, a verdict no distance measure can express and which production stores as pro/con structure. This is 15 points above the reference paper's fine-tuned embedding, achieved with zero task-specific training and a model that costs $\approx \$0.11$ per thousand statements. Inspection of B1's 44 failures shows they concentrate in genuinely ambiguous anchors (sarcasm, self-contradiction) and a few triplets whose “paraphrase” arguably flips the anchor's stance itself — i.e., partly label noise in the evaluation set rather than classifier error.

4.3 The main discovery: the summaries, not the judge, are the weak link

Under B2 — the same judge reading the 5–15-word canonical summary that production actually stores — accuracy fell 24.9 points to 70.1%. Two hundred twenty-two triplets that B1 resolves correctly fail under B2, against 4 in the other direction. The failure mechanism is consistent and visible: compression shifts the proposition. An anchor arguing that *individuals guided by doctors* should decide became the summary “The medical community should decide abortion policy”; the judge then — correctly — refused to match statements that no longer agree with the shifted text. The judge is faithful; its input is lossy.

This is the report's most actionable finding, because it is cheap to fix: the classification prompt currently renders each codebook entry as the bare summary line. Including the claim's public explanation and one exemplar member statement per line should recover most of the lost fidelity — a prompt-construction change, verifiable by re-running this benchmark.

4.4 A crowded codebook mostly inherits known losses

Under strict scoring (the match must attach to *its own* anchor's claim among ~94) condition C scores 36.0%. Decomposition shows most of the gap is not list-scale confusion: 74.4% of matches attach to *some* claim (versus 79.7% single-claim recall in B2 — so list length itself costs only ~5 points); 290 matches attach to a *different* claim, and sampling shows these are predominantly legitimate equivalents — the unconsolidated benchmark codebook contains many near-duplicate claims, in one sampled case two word-for-word identical ones. Meanwhile the critical safety property holds: distractors wrongly attaching to their own claim stayed at 10.3%, in line with B2. At production scale the judge keeps rejecting flips and keeps filing paraphrases somewhere reasonable; it inherits the summary problem of §4.3 and is penalized by duplicates that production's consolidation would have merged.

4.5 The cheap model is the right model

gpt-4o gains 1.5 points over gpt-4o-mini (96.5% vs 95.0%, $p = 0.047$) at roughly twenty times the price. Interestingly, the cheap model names the opposition more often (“opposes”: 95.9% vs 80.6%; gpt-4o prefers a cautious “none”), so mini actually yields *better* counter-edge coverage while both reject the false attachment equally well. Production's division of labor — mini decides, gpt-4o audits a 5% sample — is validated.

4.6 Per-deliberation consistency

Source deliberation	A embeddings	B1 registry	B2 summaries	D gpt-4o
Abortion survey (generation)	0.0%	98.0%	76.0%	98.0%
Abortion survey (validation)	1.0%	92.0%	79.0%	97.0%
Chatbot personalization survey	0.0%	90.9%	52.5%	90.9%
Seattle \$15 minimum wage (n=13)	0.0%	76.9%	30.8%	92.3%
Bowling Green town plan	0.0%	99.0%	93.0%	99.0%
Brexit consensus	0.0%	97.9%	68.8%	97.9%
Canadian electoral reform	0.0%	96.0%	60.0%	97.0%

Universal basic income (n=15)	0.0%	93.3%	80.0%	100.0%
Campus protests	2.0%	98.0%	68.0%	98.0%
Foreign intervention	0.0%	97.0%	62.0%	97.0%
Right to assemble	1.0%	90.0%	74.0%	94.0%

Table 2 — Triplet accuracy by source deliberation. The registry (B1) is $\geq 90\%$ on ten of eleven; the exception is the smallest and noisiest set (n = 13, containing sarcastic anchors).

5 What A2 does and does not prove

The shortlist-then-read design (A2) tied B1 exactly — because on this dataset the shortlist never filtered anything. Hard triplets are lexically close *on purpose*; every candidate sails past the similarity gate, so the reader always saw the right claim. Two consequences. First, all of the accuracy comes from the reader; the geometry contributed nothing here. Second, this dataset **cannot** test the registry's other design claim — that always reading the *full* list catches same-idea/different-words statements that a geometric shortlist would miss. Testing that requires the opposite kind of pair (low similarity, same meaning), which the registry's own production decision log accumulates over time.

6 Operational lesson: rate limits can silently corrupt fail-safe classifiers

The gpt-4o condition initially saturated the organization's 30,000-tokens-per-minute limit for that model. The production LLM wrapper retried three times within ~1.5 seconds — tuned for transient blips, not a bucket that refills over a minute — and the classifier then *failed closed*, returning “none.” By design that is the safe degradation for a live pipeline; but a failed-closed “none” was indistinguishable from an honest “none,” and 260 corrupted results entered the data before detection. The rows were purged and re-run under matched concurrency (zero corruption after). Two production fixes were merged from this lesson (commit 25f25c07): rate-limit-aware retries that honor the server's requested wait, and an explicit `failedClosed` marker so degraded verdicts can never masquerade as judgments — in a live deployment the old behavior would have surfaced as a silent burst of spurious “new claim” spawns during any dense backfill.

7 Cost

Stage	LLM calls	Estimated cost
B1 — classification, gpt-4o-mini	1,750	\$0.19
B2 — claim generation + classification	2,625	\$0.25
D — classification, gpt-4o	1,750	\$3.16
C — classification against ~94-claim codebooks	1,750	\$0.72
Embeddings (conditions A and C ordering)	≈7,500 texts	\$0.01
Entire benchmark	≈8,000	≈ \$4.30

Table 3 — Benchmark cost. Production-relevant rates: ≈ \$0.11 per 1,000 registry classifications at small codebooks; ≈ \$0.41 per 1,000 at 100-claim codebooks (gpt-4o-mini).

8 Limitations

1. **Question texts were reconstructed.** The triplet files carry no question; we wrote per-deliberation questions from the paper's descriptions. Marginal judgments could shift with different wording.
2. **The evaluation set contains label noise.** Anchors are raw participant text (including sarcasm), and a few of the paper's GPT-4o-generated “paraphrases” arguably flip stance themselves. The registry's residual errors concentrate exactly there.
3. **The recall claim is untested here** (§5): this dataset can only measure resistance to false merging, not the ability to find matches that geometry misses.
4. **Condition C conflates three effects** (summary loss, unconsolidated duplicates, true list-scale confusion); its strict score is a lower bound. A consolidated-codebook variant is the natural follow-up.
5. **Pairwise, English-only, single run.** The test isolates one decision; it does not simulate the full pipeline's dynamics (cosine passes firing first, spawning, consolidation over time). Temperature-0 runs are near-deterministic but were not repeated.

9 Conclusions and recommendations

1. **Ship the registry pass.** It is the only mechanism in the pipeline that distinguishes disagreement from agreement; the embedding passes alone would merge opposing statements essentially always (§4.1). Cost: \$0.11–0.41 per thousand statements.
2. **Fix the codebook text, not the judge.** Enrich each claim line in the classification prompt with the public explanation and one exemplar statement; the B1–B2 gap bounds the available gain at ≈ 25 points (§4.3). Re-run conditions B2 and C afterwards to verify.
3. **Keep gpt-4o-mini as the worker** and gpt-4o as the sampled auditor (§4.5).
4. **Respect per-model rate ceilings in any batch operation** (backfills, first-runs): size concurrency to the model's tokens-per-minute limit; the merged retry and marker fixes make violations visible rather than silent (§6).
5. **Follow-up measurements:** a consolidated-codebook version of condition C; and a low-similarity paraphrase benchmark built from production decision logs, to measure the recall advantage this dataset cannot show.

Reproduction: `scientific-research/20206-07-16-Claim-registry/benchmark/` — the harness imports the production classifier unmodified; all raw per-triplet decisions are preserved in append-only result files. Reference: C. Blair, A. D. Procaccia, M. Tambe, “Embeddings for Preferences, Not Semantics,” 2026 (evaluation set `hard_eval_triplets_1k.jsonl`, $n = 875$). Engineering companion with full statistical tables: `TEST_REPORT.md` in this directory.